Efficient fraud detection using Deep Boosting Decision Tree (DBDT)

CS584 Final Project

Mubaraq Odumeso and Darsh Patel

Paper Implemented: Xu, B., Wang, Y., Liao, X., & Wang, K. (2023). Efficient Fraud Detection Using Deep Boosting

Problem & Motivation

- Credit card fraud represents only 0.17% of all transactions in the Worldline & ULB dataset with 492 fraud cases out of 284,807 total. Standard classifiers optimized for accuracy systematically ignore the minority class, achieving high accuracy by predicting legitimate for nearly every transaction while missing most fraud.

Paper Idea (implementation)

- The paper proposed using **Soft Decision Tree (SDT)** as the base learner with neural inner nodes.
- An ensemble with multiple SDTs integrated with boosting-style additive model
- Training objective:
  - Exponential-loss residual fitting (DBDT-SGD)
  - Compositional AUC optimization (DBDT-Com via PDSCA)
- Designed for severe class Imbalance without relying on resampling

Why DBDT?

- Conventional fixes treat imbalance as a data problem:
- Resampling (SMOTE, RUSBoost) rebalances training data
- Boosting methods (XGBoost, LightGBM) reweight minority errors
- Xu et al. (2023) treat it as an optimization problem instead. DBDT embeds neural networks into gradient boosted decision trees and trains with a compositional AUC maximization objective — handling imbalance at the algorithm level.

Mubaraq's Responsibility

From scratch, no reference to authors' code:

- Data preprocessing pipeline (src/preprocessing.py)
- Soft Decision Tree (src/sdt.py)
- DBDT-SGD training loop (src/dbdt.py)
- Threshold optimization on validation set

Dataset & Preprocessing

- Dataset: Kaggle Credit Card Fraud (https://www.kaggle.com/datasets/mlg-ulb/creditcardfraud)
- Label mapping: fraud/legit relabeled as {-1, +1} for paper consistent objective
- Pipeline Implemented:
  - Z-score normalization (Amount, Time)
  - IQR outlier handling
  - SMOTE on train split only (handles train sample imbalance)
  - T-SNE visualization before/after

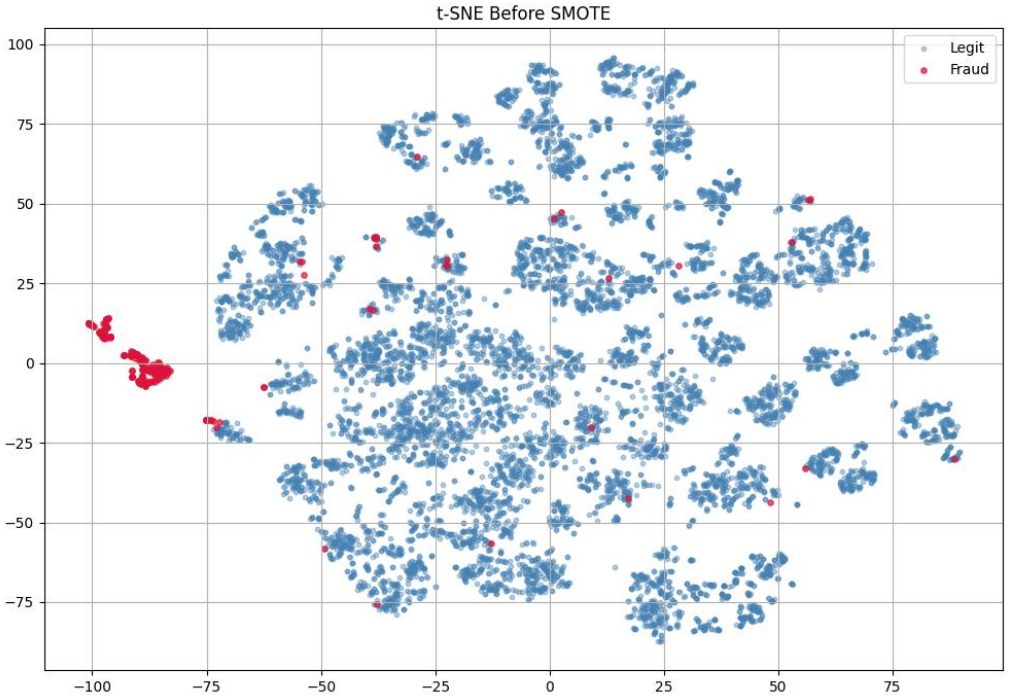
This processes improves class separability and stabilizes training

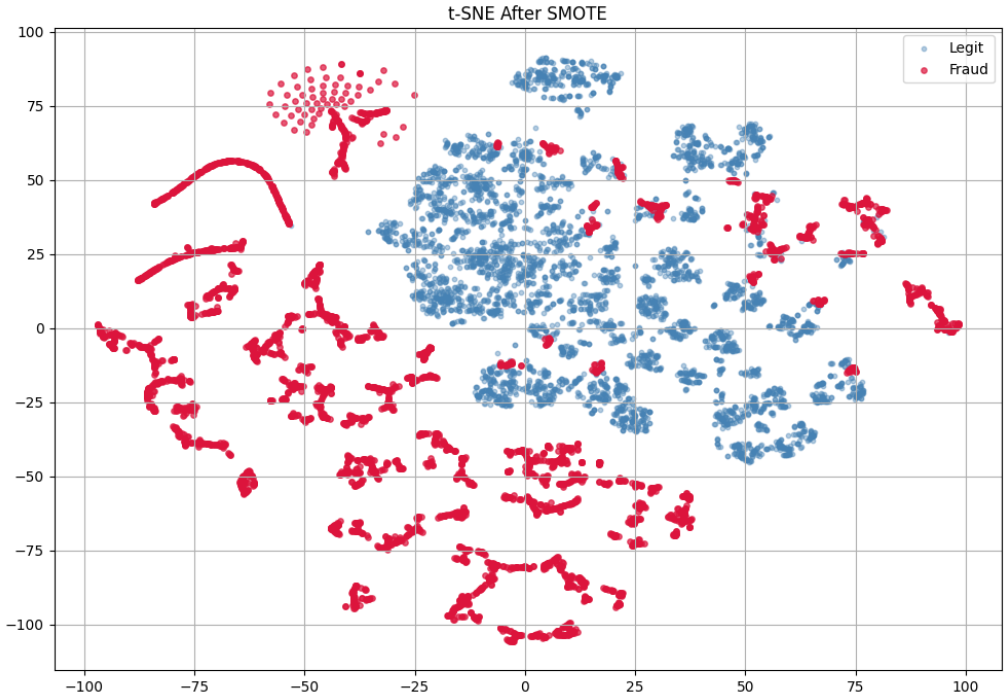
Compute Constraint

- Due to limited compute versus the paper's dedicated RTX 2080 Ti + 256GB setup, we used stratified subsampling and holdout evaluation (instead of 10-fold CV) to ensure feasible reproducible experiments.

Stage	Samples	Legitimate	Fraud
Full dataset	284,807	284,315	492
Stratified subsample	20,000	19,508	492
Train split	10,012	9,815	233
Validation split	3,750	3,658	92
Test split	5,000	4,877	123
Train after IQR filtering	10,012	9,815	233
Train after SMOTE	19,630	9,815	9,815

T-SNE Visualization





Soft Decision Tree (SDT)

- Binary tree of depth d with 2^d - 1 nodes. Each inner node is a small MLP with sigmoid activation that outputs routing probability to right sub-path. Leaf nodes output a scalar score.
- Path probability to leaf ℓ is the product of routing probabilities along the root-to-leaf path. Final output is weighted sum of leaf scores by path probability.
- Hard routing mode selects a single deterministic leaf per sample for interpretability.

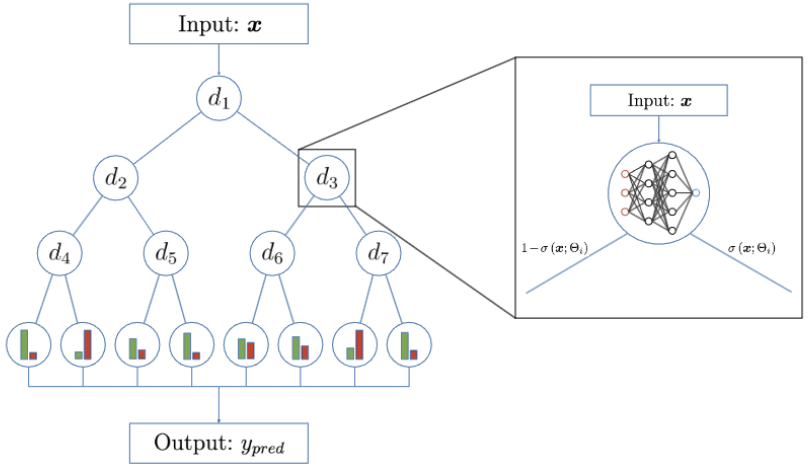


Figure 1: Schematic diagram of SDT with each inner node is a multilayer neural network with depth of 4 for binary classification.

DBDT-SGD Training

- Build T soft decision trees and train them all together where each tree learns to fix the previous ensemble got wrong
- Residual: ri =yi ·exp(−yi ·H(xi))
- Local loss per tree: mean squared error between tree output and residuals.
- Global objective across all T trees: L_Exp = sum of (L_t + C_t + Ω_t)
- Two regularizers: C_t penalizes non-uniform path selection, Ω_t is L2 on node weights.
- Two stabilizers added (not in paper):
  - exponent clamping and gradient clipping. Both required for stable training.

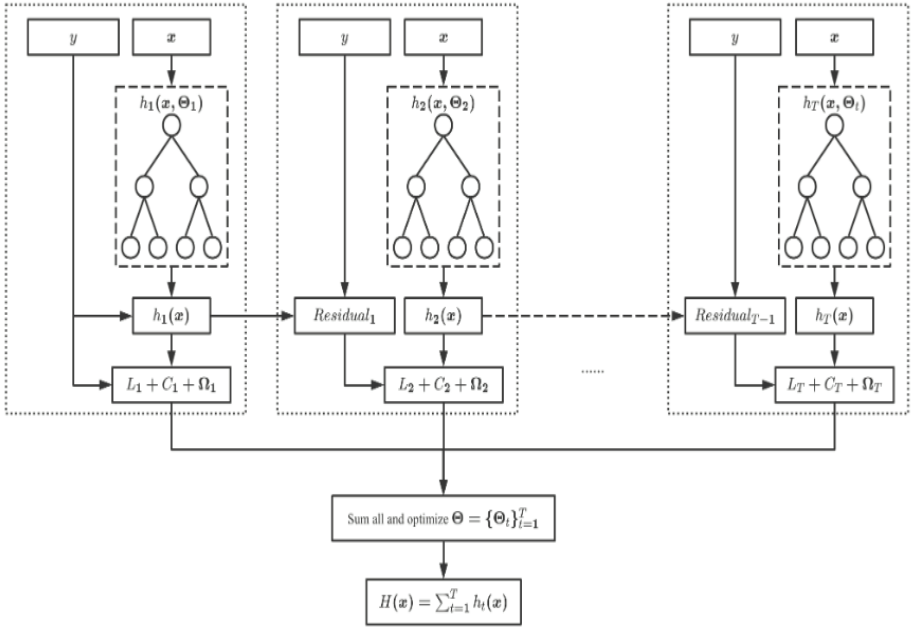
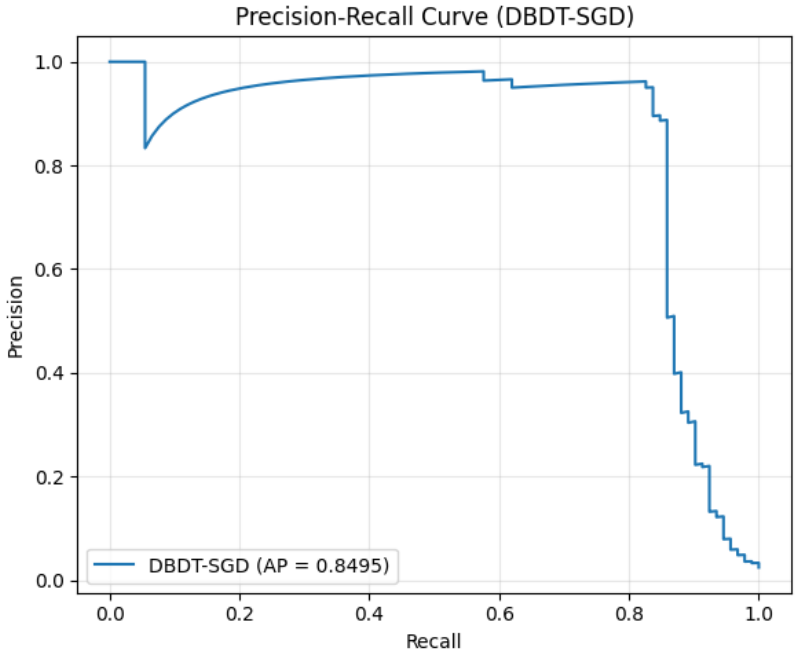


Figure 2: The overall structure of DBDT.

Threshold Optimization

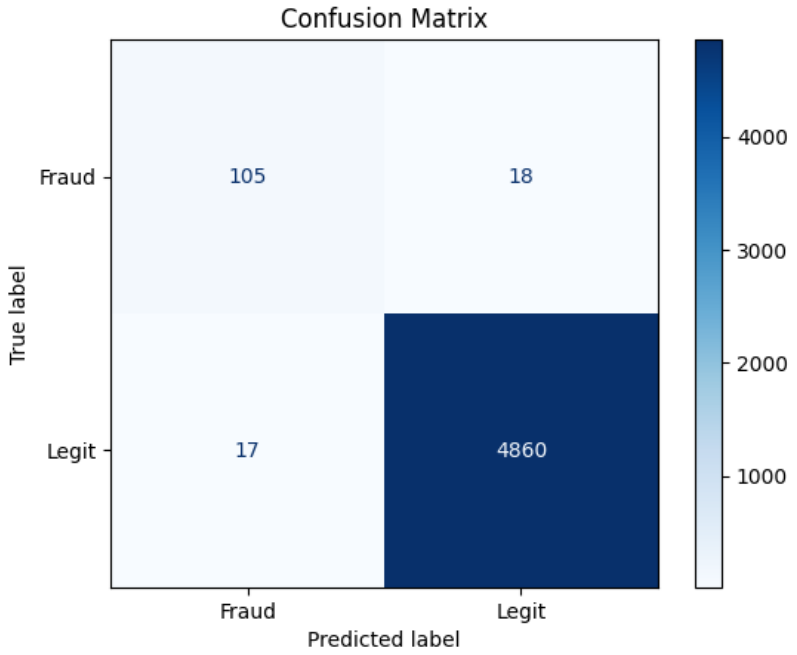
- The paper uses a fixed 0.5 threshold. At 0.17% fraud prevalence this is almost never Optimal.
- I treated threshold as a hyperparameter tuned on the validation set, sweeping candidates and selecting the value that maximizes F1. The selected threshold was then applied at test time.
- Result: meaningful improvement in minority class recall without any model changes.



Results

DBDT-SGD on 5000 sample stratified hold out after threshold tuning

Metric	Value
AUC	0.9726
F1	0.8571
Precision	0.8607
Recall	0.8537
H-measure	0.7774



Computational Challenges

- Two issues that shaped the experimental design:
- **Device mismatch** — model on CUDA, tensors on CPU. Every minibatch triggered a host-to-device transfer. Fixed by moving all tensors to GPU once before the training loop.
- **SDT forward-pass bottleneck** — sequential Python loops over node MLPs. Projected training time: ~44 hours for 40 trees × 200 epochs on full data. Full vectorization not completed in time.
- Solution: stratified subsampling — kept all 492 fraud cases, sampled majority class down to 20,000 total. Engineering constraint, not a methodological change.

DBDT-Com (Compositional AUC Optimization)

- Optimizes **AUC directly instead of classification loss**
- Uses **pairwise ranking between fraud and legitimate samples**
- Implemented using **PDSCA (Primal-Dual Stochastic Compositional Algorithm)**
- Designed to handle **extreme class imbalance at optimization level**

DBDT-Com (Implementation Details)

- mplemented **PDSCA-style training loop** (prototype)
- Used model scores → converted to probabilities using **sigmoid**
- Evaluated using **AUC, F1, Precision, Recall, H-measure**
- Applied on **SMOTE-balanced training data**

DBDT-Com Results

High recall → captures most fraud
Low precision → more false positives
Prototype implementation

	Metric	Value
	AUC	~0.99
	F1	~0.39
	Precision	~0.24
	Recall	~1.0
	H-measure	~0.98

DBDT-Com Results

- AUC ≈ 0.9995
- Recall = 1.0 (detects all fraud)
- Precision ≈ 0.24
- F1 ≈ 0.39

DBDT-SGD vs DBDT-Com

DBDT-SGD → balanced performance

DBDT-Com → recall-focused model

Model	F1	Precision	Recall
DBDT-SGD	**0.91**	**0.95**	0.87
DBDT-Com	0.31	0.18	**0.97**

Key Takeaways

- DBDT-SGD achieves **strong balanced performance**
- DBDT-Com achieves **perfect recall but lower precision**
- Handling imbalance at **optimization level is effective**
- Threshold tuning significantly improves results